

AI Engineer Take-Home - DPO Fine-Tuning with Soup

Model: Qwen2.5-1.5B-Instruct Hardware: Google Colab Tesla T4, 15,360 MiB VRAM

Stack: Soup 0.75.1, Python 3.12.3, PyTorch 2.14.1+cu130, Transformers 5.17.0, TRL 0.29.1, PEFT 0.21.1

Training: DPO; LoRA $r=16$, $\alpha=32$; batch 2; gradient accumulation 4; 1 epoch; gradient checkpointing; RAM layer streaming; no quantization.

I used [d0rj/full-hh-rlhf-ru](#) as an open-source proxy because the supplied preference data was not available. This is an important limitation: HH-RLHF conversations are not real customer-support tickets. I prepared 500 training pairs and a deterministic 50-pair held-out set. Before the final run I recorded hashes for the dataset, configuration, model weights and verification script.

1. Pre-training memory budget

I estimated memory before training. After accounting for streamed base weights, LoRA/optimizer/reference state, activations and logits, I explicitly budgeted about 2.67 GiB and predicted a peak of roughly 3.7-5.2 GiB including framework/CUDA overhead.

The estimate was too low, and the main error is identifiable. I treated logits as one fp16 tensor, approximately 2 bytes per element (~1.16 GiB). Soup's streaming pre-flight instead budgets about 14 bytes per logit element at peak because multiple logits-shaped buffers remain live during the DPO backward path. For four DPO sequences x 1,024 tokens x a 151,936-token vocabulary, this is about 8.1 GiB, close to Soup's reported 8.71 GB logits term.

Soup predicted ~10.06 GB and reported a 10.2/14.6 GB training peak. Independent one-second nvidia-smi sampling reached 14,215 MiB = 13.882 GiB. Soup reports PyTorch peak allocated memory, whereas nvidia-smi observes a broader driver-visible footprint including allocator-reserved memory and CUDA context. The larger footprint is consistent with recoverable allocation warnings; the log audit, not the memory number itself, was used to verify that training semantics did not change.

2. Silent-failure checks

Risk	Check and result	Soup?
Environment / version drift	Colab Python 3.13 resolved Soup 0.72.4 because newer releases require Python <3.13. The older build predates fixes relevant to pre-Ampere dtype handling and DPO seed propagation. soup doctor did not flag the mismatch. Rebuilt under Python 3.12 / Soup 0.75.1.	No
Wrong eval source	The dataset test split was unsuitable for Russian evaluation: over the first 2,000 rows, median Cyrillic ratio was 0.0 and none passed my ≥ 0.5 Russian-language threshold. Held-out set was carved from filtered Russian train data; train/eval overlap = 0.	No
Prompt truncation	41/500 early prompts exceeded the 512-token cap. Because raw prompts end with the final Ассистент: marker, keep_start truncation would necessarily remove that marker and could remove part of the latest user turn. Final max prompt length: 495.	No
Tokenizer boundary	A previous run produced TRL prefix warnings. Added exact token-prefix R9 filtering, removed four problematic candidate rows; final train/eval audit: 0 mismatches.	Runtime warning only
HF cache / materialization	Dry-run passed, but live training initially failed on cached snapshot layout. Materialized the exact pinned model locally and verified its weight hash.	Live run only
GPU/OOM fallback	Final run emitted five CUDA allocation warnings. Raw logs showed no traceback, skip, retry, reduced/halved batch, auto_find_batch, NaN/inf or meta tensors. Training completed 63/63, epoch 1/1, exit 0.	Warnings only; audit manual

The final adapter contained 392 tensors, including 196/196 non-zero LoRA-B tensors, no .inner. keys, no meta tensors, and all 392/392 tensors reloaded exactly. A remaining limitation is train/serve formatting: HH examples were trained and scored as raw text, not through the Qwen chat template.

3. Verification: did the model actually learn?

Falling training loss is not sufficient proof: it is measured on fitting batches, can improve through memorisation or shortcuts, and does not prove that the saved adapter reloads correctly or improves held-out behaviour. Before the final run, I froze the verification script, thresholds and held-out set and recorded their hashes. Baseline summary metrics reproduced exactly across two independent Colab sessions (mean margin 15.9766, median -5.7593, chosen > rejected 0.46, mean margin/token 0.1779); within-session repeat evaluation had zero numerical difference.

- baseline mean preference margin: 15.9766; tuned: 16.1983
- mean held-out Delta: +0.2217 nats; bootstrap 95% CI: [-0.4219, +0.8899]
- fraction improved: 54%; sign-test $p=0.336$; chosen > rejected: 0.46 -> 0.46
- $\text{corr}(\text{Delta}, \text{response-length difference})$: 0.499

The adapter therefore changed parameters and outputs, but the held-out test did not show convincing preference improvement. A random same-scale adapter control also failed the trained-model criterion as expected.

The update was small on train as well: the final logged training reward margin was about 0.037, or roughly 0.37 nats on the underlying DPO log-ratio scale at $\beta=0.1$, the same order as the held-out Delta of +0.22. Aggregate DPO loss remained near $\ln(2)$ at about 0.684. I interpret this as a small update that $n=50$ cannot confidently distinguish from noise, rather than strong learning followed by failed generalisation.

soup ship also returned DON'T SHIP, but its task metric was non-discriminative (0.0000 -> 0.0000), so I do not count it as independent task evidence. Its useful contribution was the forgetting check: all eight bundled general benchmarks stayed within the 5% regression threshold, so no catastrophic forgetting was detected. No noise floor was measured. Ship evaluation reported bf16 on the T4 while training used fp16, so I treat base-vs-tuned results as internally comparable, not as a reproduction of training numerics.

What this verification does not prove: generation quality; correctness of preference labels; Qwen chat-template behaviour; forgetting outside Soup's bundled suite; transfer to real support tickets; per-layer gradient correctness; or small effects that $n=50$ is underpowered to detect.

4. Verdict - DON'T SHIP

The final run is technically valid and the adapter demonstrably trained, but I cannot show that it materially improved the target preference task. The held-out confidence interval crosses zero, the sign test is not significant, and the small shift is moderately correlated with response length. I would therefore not ship this adapter. Before reconsidering, I would use real support-ticket preference data, separate validation and untouched test sets, align training/serving prompt formats, choose the training budget using validation only, evaluate more than one seed, and investigate length sensitivity.

Surprise / concern. Several individually successful checks were insufficient: dry-run did not expose the cache-layout failure, successful completion did not explain CUDA warnings, and falling loss did not imply held-out improvement. My largest technical miss was the memory estimate. I also did not run a resident-vs-streamed gradient-parity test on this exact T4/Qwen setup; my verification establishes aggregate parameter/output change, not per-layer gradient exactness.

AI assistance. I used AI tools for debugging unfamiliar Soup/TRL behaviour, code review and verification design. Suggestions accepted only after checking included the dataset/filtering approach, model/no-quantization setup, choosing Soup 0.75.1 after reviewing its changelog, and the baseline/Delta/control-based verification design. AI also suggested Python 3.13 as the cause of stale Soup resolution; I verified this against package Python-version requirements. I corrected AI suggestions when evidence contradicted them: the initial held-out choice did not check language suitability, and the first verification encoder tokenised prompt/completion separately instead of matching TRL concatenated tokenisation. Other runtime issues I reproduced and verified included pre-Ampere dtype behaviour on T4, data doctor refusing DPO format, token-boundary mismatches requiring R9, and the HF cache-layout failure. Final claims are based on executed code, hashes, raw logs and saved artifacts.